

Executive Summary

This document outlines a comprehensive UI brief for the **Mastercard Innovation Challenge: AI Defense Lab for Payment Security**. It describes an end-to-end *red-team/blue-team* system: first **Identify** new GenAI-powered payment fraud attacks, then **Generate** realistic synthetic attack data, and finally **Defend** by training a detection model. We propose a hybrid web UI combining a single-page “story” and multi-page details. The brief covers system goals, over 25 fraud attack vectors with real-world examples, data schemas, evaluation metrics (precision/recall/F1/AUC), feedback-loop training, API design, security/privacy, deployment, and required artifacts (code repo, walkthrough doc, web prototype). The UI uses best practices from top design systems (Material Design, Ant, Coinbase CDS, etc.) [17†L143-L152] [30†L268-L275] . Diagrams (Mermaid) illustrate the architecture and user flows. A concrete implementation roadmap and testing plan are included, along with example payloads. This deep-dive ensures clarity and completeness for driving an auto-generated UI mockup.

Project Overview

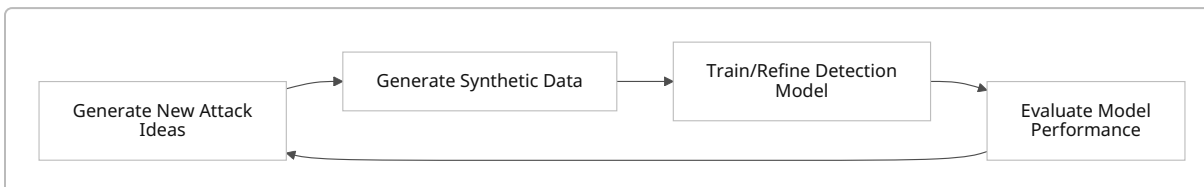
The goal is a **closed-loop AI Defense Lab for payment fraud**. Participants must **(1) identify** emerging GenAI-powered fraud schemes, **(2) generate** simulated transactions that mimic those attacks, and **(3) defend** by building ML classifiers to detect them. The strongest solutions link these pillars: generated attacks form training data that improve the defense, and defense gaps inspire new attack ideas. This mirrors an *adversarial ML* cycle. Our UI/UX will reflect this story-driven workflow: an overview narrative guides users through Identify→Generate→Defend steps, with options to “drill into” detailed tools and dashboards. We follow best practices from modern fintech/SaaS UIs: modular components, clear charts, and responsive design [8†L292-L299] [20†L114-L123] .

Goals & Approach

- **Identify (Ideation):** Survey and catalog as many plausible GenAI-driven payment fraud scenarios as possible. Consider social engineering (email, voice), technical exploits, new rails (real-time, digital), and multilayer schemes. Each idea should be grounded in real payment flows.
- **Generate (Simulation):** Implement agents or algorithms that **simulate those fraud scenarios** at scale. This may involve using LLMs to write scam emails, GANs to create synthetic user data, or code to simulate automated fraud (bots, phishing campaigns). Fidelity is key: generated transactions and attack artifacts should look indistinguishable from real data. For example, fake invoices should match genuine templates, phishing texts should mimic company communications, and synthetic identities should follow valid patterns. We will use synthetic data tools (GANs, LLMs, etc.) while ensuring privacy (see Security).
- **Defend (Detection):** Train ML models (e.g., binary classifier) to flag generated attacks. Evaluate using **precision, recall, F1, ROC-AUC/PR-AUC**, tuning to meet business needs (e.g. high recall at acceptable false-positive rates) [39†L77-L85] [40†L41-L50] . The UI will display key metrics (F1,

precision/recall curves) and examples of flagged transactions. Importantly, this is iterative: missed attacks identified by the model (false negatives) feed back into “Identify” for new ideas.

These steps form a feedback loop: *attack ideas* → *synthetic data* → *trained model* → *evaluation* → *refined attack ideas*. This loop is visualized below:



Fraud Attack Taxonomy (25+ Examples)

We categorize potential GenAI-driven payment fraud attacks by technique and channel, citing real-world analogues where possible:

- **Synthetic Identities:** AI blends real and fake data to create new identities. *Example:* GAN-generated identity profiles used to open accounts for money laundering. Synthetic ID fraud is known to scale with AI [36†L129-L137] .
- **Deepfake Documents & IDs:** AI alters or fabricates identity documents. *Example:* Deepfake driver’s license images bypassing KYC checks.
- **AI-Phishing (Email/SMS):** LLMs craft highly personalized phishing emails or texts. *Example:* GenAI finds a victim’s recent social media posts to generate a convincing invoice email. Massive-scale, multi-language spear-phishing is now feasible [36†L120-L127] .
- **Voice Deepfake (Vishing):** AI-generated or cloned voices impersonate executives or officials. *Example:* Deepfake CEO voice calls an employee to authorize a fund transfer. Group-IB reports voice cloning in seconds and billion-dollar losses [59†L302-L310] .
- **Image/Video Deepfake Social Engineering:** AI creates realistic images or videos. *Example:* A fake video message from “support staff” instructing a user to install malware or send money.
- **Chatbot Scams:** Malicious chatbots (e.g. on websites or messaging apps) using LLM to interact and extract credentials or payment info. *Example:* A “customer support” chat that uses context to mislead customers.
- **Fake Invoices/Requests:** LLMs generate fraudulent invoices mimicking vendor style. *Example:* An AI produces a convincingly formatted payment request to a company, causing an unauthorized transfer [36†L139-L144] .
- **Smishing (SMS Scams):** Text messages generated by AI, often tailored and multilingual, linking to phishing sites.

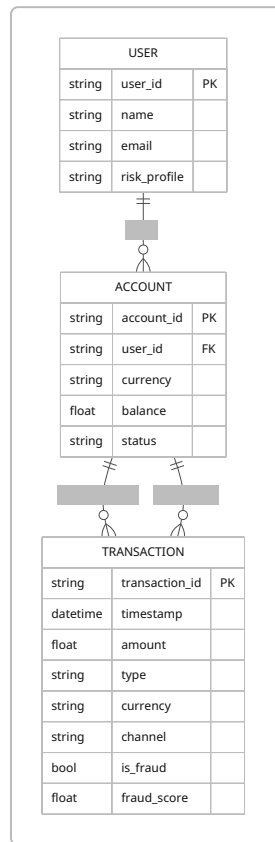
- **Malware with AI Payloads:** Malware that uses generative AI to evade detection. E.g., code polymorphism or generating novel phishing URLs.
- **Bot-driven Fraud:** Networks of bots (controlled by AI) making small test transactions or applying strawman accounts.
- **Social Media Influence:** AI-driven disinformation to manipulate stock or asset prices, indirectly affecting payment risk.
- **Account Takeover (ATO):** Using AI to guess or reset passwords (via mass credential stuffing, or LLM social inference).
- **Unauthorized Micro-Transactions:** AI automates many micro-charges to avoid detection (small frauds under radar).
- **Triangulation Fraud:** A buyer uses stolen card via AI-generated marketplace or dropshipping scheme. AI can optimize the flow of goods/payments to avoid obvious patterns.
- **Money-Mule Networks:** AI coordinates networks of mule accounts, optimizing transfer timings and amounts.
- **Crypto-Mixing Attacks:** AI orchestrates cryptocurrency laundering through multiple mixers or coins, creating complex flow patterns.
- **Generative SMS Social Pressure:** AI texts victims, pretending to be from bank fraud team, to get credentials.
- **Insider Threat Amplification:** A malicious insider uses AI to craft credible internal messages or alter systems.
- **Adaptive Attack Campaigns:** Generative AI auto-tunes attacks in real-time based on defender responses.
- **Election/Social Payment Scams:** Exploiting social movements, AI generates fake donation drives or relief funds.
- **Fake Reviews or Ratings:** Using GenAI to pump fraudulent reviews to boost deceitful fintech products that steal user funds.
- **Adversarial Model Attacks:** Crafting inputs that specifically break ML-based detectors (though defense loop should catch these over iterations).

Real-world evidence: FedPayments notes that “AI empowers criminals to create more convincing attacks at scale, including phishing, synthetic IDs, deepfakes (voice/images), and fraudulent documents” [36†L70-L74] . In Asia, deepfake fraud attempts surged 194% in 2024 [59†L302-L310] . Such context underlines

why we aim for **breadth (25+)** and depth. Each attack vector above would include an example scenario in our UI brief (and potential indicators for detection).

Data Models & Synthetic Transactions

To simulate payment data, we define data schemas matching real systems. A simplified model:



- **Entities:** Users have Accounts, which engage in Transactions. Transactions include payer/receiver accounts, amounts, timestamps, channels (e.g. ACH, card, wire), and flags (fraud).
- **Synthetic Data:** We will generate transactions and related user/account profiles. For example, a synthetic transaction payload might look like:

```
{
  "transaction_id": "txn_002",
  "user_id": "usr_100",
  "amount": 9999.99,
  "currency": "USD",
  "timestamp": "2026-08-28T16:45:00Z",
  "merchant_category": "electronics",
  "location": {"country": "BR"},
  "metadata": {
    "payment_method": "wire",
```

```

    "customer_ip": "192.0.2.5"
  }
}

```

This JSON illustrates the fields used in simulations. The **defense model** would output a JSON like:

```

{
  "transaction_id": "txn_002",
  "is_fraud": true,
  "fraud_score": 0.92,
  "fraud_reasons": ["amount_anomaly", "unusual_geography"],
  "model_version": "GenAI_Detector_v1",
  "analysis": "Large wire transfer to a foreign country flagged by pattern X."
}

```

This output includes classification (`is_fraud`), a score, and reasons. These schemas will be documented and used consistently in the UI prototype and API.

Detection Metrics & Evaluation

Fraud detection is a **binary classification** with heavy class imbalance (fraud is rare). We will track standard metrics:

- **Precision:** fraction of flagged transactions that were truly fraud [39†L77-L85] . High precision means few false alarms.
- **Recall (Sensitivity):** fraction of actual fraud cases detected [39†L103-L112] . High recall means few missed frauds.
- **F1-score:** harmonic mean of precision and recall [39†L123-L132] , useful to balance both.
- **ROC-AUC:** ability to discriminate fraud vs non-fraud across thresholds [40†L13-L21] .
- **PR-AUC:** precision-recall curve area, often more informative under imbalance [40†L41-L50] . (A high ROC-AUC can mask poor precision/recall on the minority class; PR-AUC focuses on fraud detection quality.)

Our UI will display e.g. an ROC curve and precision-recall curve. We will choose an operating point (threshold) guided by business constraints (e.g. “catch $\geq 80\%$ of fraud” while keeping false alerts below X%). As Cesar Soto’s fraud metrics guide notes, systems often **set a recall target and maximize precision under that constraint** [39†L119-L122] . In evaluation, we’ll hold out a test set of synthetic attacks, report all metrics, and possibly compare with baseline detectors.

Key targets: We aim for high recall (80–90%) with as high precision as possible. We will also monitor false positive rate (FPR), since too many false declines hurt customers [39†L142-L150] . In sum: “Precision@Recall” curves, F1 scores, and AUCs will be presented. We’ll also log confusion matrices and set up dashboards to compare model versions.

Closed-Loop Training Pipeline

The system is an iterative ML pipeline. Conceptually: generate synthetic data → train model → evaluate → refine attacks. Implementation plan:

1. **Data Ingestion:** Simulated transactions (genAI attacks + legit mix) are fed into a training set.
2. **Model Training:** An ML pipeline (e.g. Python with scikit-learn/XGBoost or PyTorch) trains a fraud classifier. Possibly use cross-validation.
3. **Evaluation & Logging:** Performance metrics are computed. Misclassified cases (false negatives) are flagged.
4. **Attack Feedback:** Review failure cases to brainstorm new or improved attack simulations. This feeds back into **Identify** and **Generate** to enrich the next data batch.

This loop is managed by the UI and backend integration. We visualize it with a flowchart (above). Practically, a continuous integration setup could retrain on new synthetic data and redeploy the model.

The UI prototype will simulate this pipeline: for example, a “Retrain Model” button might ingest the latest generated transactions and update the dashboard metrics. Metadata (model version, training data size, time) will be shown. We’ll also provide pseudo-code for API endpoints: e.g. `POST /simulateAttack`, `POST /scoreTransaction`, etc., enabling integration in a real system.

API & Integration Points

Our design assumes a **service-oriented architecture**. Key APIs include:

- **Transaction Ingestion API:** accepts incoming payments (JSON). This could be connected to payment rails or webhooks.
- **Simulation API:** to trigger generation of attack data (e.g. `POST /simulate?type=phishing_email`).
- **Detection API:** given a transaction JSON, returns fraud score and label (`POST /detect`). This can be integrated into payment processing flows.
- **Admin/Config APIs:** to define new attack scenarios, retrain models, adjust thresholds.

We assume REST+JSON interfaces. Internally, an **event-driven pattern** is natural: each payment generates an event placed on a message bus (Kafka or Pub/Sub) consumed by the detector [57†L401-L408]. This decouples payment processing from fraud checks – for example, payments proceed even if detection is temporarily offline [57†L401-L408]. A cloud-native pipeline (e.g. Kafka + Spark or Google Pub/Sub + Dataflow) can provide scalability [55†L518-L525]. For instance, Google’s Pub/Sub + Dataflow example shows publishing test transactions and scoring them in real time [55†L488-L497].

We’ll detail integration points: e.g. connecting to real-time networks (card/ACH) versus batch processing. We’ll recommend using standard OAuth2/JWT for API auth, and webhooks for alerts (e.g. push a JSON message to a “fraud-alerts” topic if `is_fraud=true`). A small sequence diagram or flowchart may illustrate a transaction lifecycle from arrival → scoring → action.

Security & Privacy Considerations

- **Data Privacy:** All synthetic data must avoid real PII. We will use GAN/LLM to *simulate* realistic personal info without using actual customer data [33†L123-L127] . This protects user privacy and compliance (e.g. GDPR, PCI).
- **Encryption & Storage:** Any sensitive fields (account numbers, user IDs) should be tokenized or hashed in storage. Transport-level encryption (HTTPS/TLS) is mandatory for all APIs.
- **PCI DSS Compliance:** If integrating real payment data, use secure elements (e.g. Stripe Elements) to handle card input, so raw card data never touches our server [8†L292-L299] . This offloads compliance and embeds secure UI components.
- **Access Control:** Role-based access will ensure only authorized users (fraud analysts vs admins) can view/edit system parts. Use multi-factor auth for admin interface.
- **API Security:** Rate-limit APIs to prevent abuse. Validate all inputs rigorously to prevent injections.
- **Audit Logging:** Every decision (flagging a transaction) should be logged with a timestamp and model version. Event-driven design inherently provides an audit trail [57†L412-L418] .
- **Adversarial Robustness:** Since attackers may attempt to poison our system, logs of training data and model behavior will be kept for analysis.
- **UX Privacy:** The UI will not display any real sensitive data. Synthetic names and masked account numbers (like "Acct•••1234") will be used in examples.

Deployment Feasibility

We assume cloud deployment (AWS/Azure/GCP) for scale. The pipeline (described above) is known to support high throughput: e.g., Google's Pub/Sub+Dataflow for streaming payments [55†L518-L525] . Modern databases or data warehouses (e.g. BigQuery, Snowflake) can store the transaction history. The ML model can be served via a scalable API (e.g. AWS SageMaker endpoints or Flask+Gunicorn behind an autoscaling group). We note that generative components (LLMs/GANs) may require GPUs or cloud ML APIs (e.g. Azure OpenAI).

Latency: Real-time payment networks require sub-second scoring. We'll propose a lightweight model (e.g., XGBoost) for real-time inferences, with batch retraining offline. If latency is tight, we can pre-compute risk scores for high-speed channels.

Monitoring: Health checks and logging (e.g. CloudWatch, ELK) will track system performance. The UI can display system status (e.g. "Backend connected", "Last transaction scored at...") to reassure users. Overall, the architecture (microservices + event-driven) aligns with cloud-native fintech practices [57†L401-L408] [55†L518-L525] .

Submission Artifacts

Per the challenge, the final deliverables include:

1. **Code Repository:** A GitHub repo with runnable code for Identify (attack definitions/generators), Generate (simulation scripts), and Defend (detection model + training). It should be well-documented and containerizable. We will reference this in the UI as "View Code on GitHub".

- 2. **Solution Walkthrough (.docx/.pdf):** A document explaining our approach, attack list, simulation methods, model design, and feasibility. We'll ensure it includes all technical details.
- 3. **Working Web Prototype:** A demo website (hosted or Dockerized) showing the UI outlined below: interactive screens for the closed-loop. The UI must reflect the architecture and use real (synthetic) data examples.

The UI design below will inform the prototype. We assume unspecified choices (framework, etc.) are flexible, but we'll note examples (e.g. React+MUI, Python Flask).

UI Design & Layout

We propose a **hybrid layout** combining a single-scroll narrative with multi-view deep dives. This takes inspiration from modern sites (e.g. Apple, Stripe docs, BMW online report) where a storytelling homepage leads into interactive pages [44†L84-L91] [44†L159-L163] .

- **Single-Scroll Narrative:** The landing page begins with a big visual (e.g., stylized graphic of fraud/ security) and introduces the challenge's concept. As the user scrolls, sections cover *Identify*, *Generate*, and *Defend*, each with illustrations or charts. Scroll-triggered animations ("scrollytelling" [44†L84-L91]) and clear headings make this engaging. For instance, an animated flowchart may draw the closed-loop as you scroll. This page ends with a call-to-action (CTA) to explore details or login to the app. This narrative page ensures new visitors immediately grasp the big picture.
- **Multi-View/SPA:** Upon interaction (clicking a CTA or menu), the site transitions (in-app) to a detailed dashboard or tool. For example, clicking "Learn about Attacks" scrolls the page to a deeper section, or better, opens a new panel / route without full page reload (SPA-style). We can use modals or tabs for drill-down content. This maintains the narrative's continuity while enabling functionality.

We compare layout options in the table below:

Layout Option	Description	Pros	Cons
Single-page narrative	One long scroll page telling the story of Identify→Generate→Defend with sections and visuals.	Highly engaging story flow [44†L84-L91] ; controls pacing; seamless on desktop/mobile.	Harder to implement complex interactions; SEO & bookmarking limited.
Multi-page app (SPA)	Traditional dashboard with separate pages (e.g. "Attacks", "Simulations", "Dashboard").	Easy to scale features; better navigation/SEO; modular.	Can feel disjointed; user may miss narrative context.
Hybrid (suggested)	Landing narrative + in-app views.	Combines best of both: engaging intro + detailed tools [44†L159-L163] .	Slightly more complex routing; care needed to maintain flow.

We favor **Hybrid**: a scroll-driven intro that segues into a Single-Page App. For example, BMW's report splits narrative vs detail sections so different users find what they need [44†L159-L163] . Our UI will similarly allow jumping from narrative sections into data-driven pages.

Screen List & Navigation

Based on this layout, key screens (with rough components) include:

- **Home (Narrative Landing)**: Full-width hero banner, followed by vertical “story” sections (Identify, Generate, Defend). Each section has an illustration/animation, brief text, and a small chart or bullet of results. At the bottom, quick links or buttons to “View Attack Library”, “Run Simulation”, “View Dashboard”.
- **Attack Library**: A searchable/filterable list of fraud attack types (25+). Each item shows name, category, example icon. Clicking an attack opens **Attack Detail** (modal or page).
- **Attack Detail**: Describes one attack scenario (text, examples). Includes a “Simulate Attack” button. Possibly a small embedded example of a scam email/video snippet.
- **Simulation Console**: UI to configure and run simulations. E.g., select attack type, number of transactions, etc. Shows progress and outputs sample generated data in a table.
- **Detection Dashboard**: Main monitoring panel. Shows key metrics (accuracy, F1, ROC curve, fraud count). Contains filters (by date, attack type). Displays charts (see next section). Table of recent flagged transactions with details (ID, amount, reason).
- **Transaction Detail**: On clicking a transaction in table, a modal/pane with full details (like API output JSON, chain of detection logic).
- **Model Training/Settings**: Page to view training logs, trigger retraining, and adjust parameters (thresholds, feature weights).
- **User Account/Profile**: (if needed) for settings, auth. Basic profile info.

Navigation Pattern: A top navigation bar (or sidebar on desktop) lists major sections: Home, Attacks, Simulate, Dashboard, etc. On mobile, a hamburger menu. The home navigation scrolls internally (“hero link”), while the others route (client-side) to SPA pages. We will ensure “Back to top” or “Back to Home” links for context. Flowchart:

```
flowchart LR
    Home -->|Click "Attacks"| AttackList
    Home -->|Click "Dashboard"| Dashboard
    AttackList -->|Select Attack| AttackDetail
    AttackDetail -->|Click "Simulate"| SimulationConsole
    Dashboard -->|Click TX ID| TransactionDetail
    Dashboard -->|Click "Retrain"| ModelTraining
    style Home fill:#f9f,stroke:#333,stroke-width:1px
```

Accessibility is crucial: use semantic HTML, ARIA labels, focus indicators, and keyboard navigation. For example, all buttons will have descriptive `aria-labels`. Coinbase's design system emphasizes clear labels and keyboard support [22†L157-L160] , which we will adopt. Color contrast will meet WCAG 2.1 AA standards.

Component Inventory

We will reuse and adapt components common to fintech/SaaS UIs. Key components (with examples from popular design systems) include:

- **Layout/Containers:** Header/NavBar, Sidebar (responsive), Cards, Grid system (e.g. 12-column).
- **Navigation:** NavBar with menu, side drawer, Breadcrumbs (for Drilldowns).
- **Form Controls:** Buttons (primary, secondary, danger), Text fields, Dropdowns, Checkboxes, Radio, Date pickers (AntD/MUI have these) [46†L45-L53] [30†L268-L275] .
- **Tables:** For listing attacks and transactions (sortable, pageable). (See Salesforce Lightning and Carbon for data table examples) [30†L230-L239] .
- **Charts/Graphs:** Line, Bar, Pie, Heatmap, Network/Sankey (via D3, Chart.js, or AntV) to visualize fraud trends [20†L114-L123] . Carbon charts cover many types [30†L183-L190] .
- **Visualizations:** Sparklines (Trend mini-charts), Meters (gauge), KPI cards.
- **Modals/Popovers:** For details, confirmations. E.g. confirm alert send, show JSON payload.
- **Notifications/Alerts:** Top-of-screen or in-dashboard banners for system alerts (e.g. “Model retrained successfully”) and in-UI highlights for flagged transactions.
- **Avatars/Icons:** Small icons or avatars for accounts/merchants. (Use icon font or SVG library).
- **Typography:** Heading (H1–H4), body text, code blocks (for JSON examples). We'll use a clean font (e.g. Inter or Roboto).
- **Colors & Themes:** Define color tokens (see below).

Table of component libraries vs features:

Library/ System	Tech	Components	Theming/Extensibility	Notes
Material UI (MUI) [46†L45-L53]	React	~100 components (Material)	Rich theming (palette, typography); widely used; active community.	Implements Google Material Design.
Ant Design [30†L268-L275]	React	60+ components + Pro pages	Themeable via Less/ Semantic tokens; TypeScript support.	Enterprise focus; many complex widgets.
Carbon Design	React/Web Components	Components + rich data viz [30†L183-L190]	Tokenized (colors/ spacing); highly accessible.	IBM's enterprise system (finance- focused).
Coinbase CDS [22†L146-L154]	React/Web	100+ components (incl. charts)	Design tokens for colors/spacing; themable.	Built for crypto apps; open- sourced.
Shopify Polaris [17†L163-L172]	React	Buttons, Forms, Layout, etc.	Comprehensive style/ content guidelines.	Emphasizes UX writing (tone, messaging).

Library/ System	Tech	Components	Theming/Extensibility	Notes
Radix/shadcn (Tailwind UI)	React	Primitive & stylized components	Fully code-ownership; theme via CSS vars.	Highly customizable; emerging popularity.

These examples show that open-source UI libraries provide ready-made elements. We assume using React (though Vue or Svelte could also fit) with one of the above. For data viz, libraries like D3 or ECharts could be used as needed (Carbon has chart guidance [\[30†L183-L190\]](#)).

Visual Design & Style (Figma-Ready Specs)

We will define a cohesive design language with tokens for colors, typography, and spacing:

- **Colors:** A primary palette (e.g. deep blue for security), secondary (gold/orange accent), neutrals (grays for UI background/text). For instance:
 - `Color.Primary = #0050C8` (Mastercard blue theme)
 - `Color.Secondary = #FDBB30` (gold accent)
 - `Color.Background = #F0F2F5` , `Color.Surface = #FFFFFF` , `Color.TextPrimary = #333333` .
 - `Success = #28A745` , `Warning = #FFC107` , `Error = #DC3545` (bootstrap-like for statuses).
- **Typography:** Base font size 16px (Body), scale headings (e.g. H1=32px, H2=24px, H3=20px). Line height ~1.5. Use a clean font stack (Inter, sans-serif).
- **Spacing:** An 8px (or 4px) grid system. Tokens like `Spacing.SM = 8px` , `MD = 16px` , `LG = 24px` , `XL = 32px` . All margins/padding in multiples of these.
- **Corner Radius:** e.g. `BorderRadius = 4px` for inputs/cards.
- **Elevation/Shadows:** if needed, subtle shadows for cards/modal (MUI style).

These design tokens (in Figma variables or CSS variables) ensure consistency. For example, Coinbase CDS describes a modular token system for colors/typography [\[22†L165-L173\]](#) , and Material You uses dynamic theming [\[17†L143-L152\]](#) .

Responsive design: We'll use breakpoints (e.g. 600px, 960px, 1280px) to adjust layouts. On mobile, the narrative scroll still works (stacked sections). Tables become vertical lists, charts simplify (e.g. hide less-critical axes). Navigation collapses to a hamburger. Accessibility: provide text alternatives for all non-text content. For charts, include data tables or descriptions as needed.

Data Visualization Guidelines

Aligning with dashboard best practices [\[20†L114-L123\]](#) :

- **Chart Types:** Choose per data: Line charts for time-series trends, Bar charts for category comparisons, Pie/Donut for part-to-whole (e.g. fraud by type), Scatter for correlations (e.g. amount

vs time), Heatmap for time-of-day by day patterns, Sankey or network diagrams for transaction flows. Carbon's guidance suggests picking chart type by purpose [30†L183-L190] .

- **Design:** Always show labels/units. Avoid misleading 3D effects. Use color-blind-friendly palettes (blue/orange over red/green). Do not rely on color alone; use patterns or annotations for emphasis [20†L117-L120] .
- **Interactions:** Allow tooltips, zoom, and filters on charts. For example, a timeline slider to filter transactions or toggling attack types.
- **Examples:** The UI mockups will include sample charts: e.g. an F1-score timeline, pie chart of fraud distribution, or line chart of fraud rate over time.
- **Compliance:** For accessibility, every chart needs a text summary or ARIA description. Users should be able to interpret data without color (e.g. with pattern fills).

Example JSON Payloads

To guide the UI generation, here are concrete payload examples. **Simulated attack transaction:**

```
{
  "transaction_id": "txn_AIphish_001",
  "timestamp": "2026-08-28T15:30:00Z",
  "amount": 1250.00,
  "currency": "USD",
  "from_account": "acct_1234",
  "to_account": "acct_scam9876",
  "merchant": "ACME Corp",
  "category": "Electronics",
  "description": "Invoice #45021",
  "tags": ["ai_generated_phishing", "invoice_fraud"]
}
```

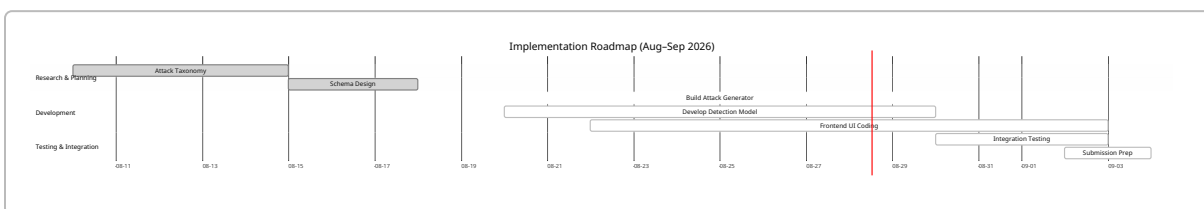
Detection output:

```
{
  "transaction_id": "txn_AIphish_001",
  "is_fraud": true,
  "fraud_score": 0.92,
  "detected_at": "2026-08-28T15:30:01Z",
  "reasons": {
    "pattern": "invoice_template_mismatch",
    "sender_history": "unusual_recipient",
    "nlp_flags": "suspicious_phrasing"
  },
  "model_version": "GenAIDefender_v1"
}
```

These examples will be used in the prototype UI (e.g. shown on Transaction Detail). JSON viewers in the UI should use monospace font and syntax highlighting for readability.

Frontend Implementation Roadmap

Below is an example Gantt chart outlining tasks and timeline (over roughly 3–4 weeks up to the Sept 1 deadline):



Each task can further break down (e.g. UI Coding splits into Layout, Components, Data Fetching). We plan agile sprints: the first weeks finalize attack definitions and data models; mid-phase builds generators and model; final week focuses on UI polish and system testing.

Testing & Validation

Our testing plan includes:

- **Unit & Integration Tests:** Automated tests for data generation scripts and ML model code. For UI, component snapshot tests (if React) and linting.
- **Simulation Checks:** Validate that synthetic data matches real distributions (mean amounts, frequencies). Use statistical tests on generated vs baseline data.
- **Evaluation Tests:** Measure model metrics on held-out synthetic fraud and legitimate samples. Ensure precision/recall meet targets before release.
- **Load/Performance:** Simulate high transaction rates (hundreds/sec) to test system throughput. Ensure detection API latency under 200ms.
- **Security Tests:** Penetration testing for APIs (injection, auth bypass).
- **Usability/User Tests:** Informal reviews where users follow tasks (e.g. “simulate a phishing attack, find its detection”). Gather feedback on clarity of flows. Accessibility audits (e.g. Lighthouse, manual keyboard navigation).

After deployment, we will monitor logs for unexpected behavior (e.g. model drift). The feedback from Defense (e.g. false negatives) will dynamically generate new test cases in future cycles.

References

Leading design systems and UX resources informed this brief: Google’s Material Design tokens [17†L143-L152] , Alibaba’s Ant Design components [30†L268-L275] , IBM’s Carbon data-visualization guidelines [30†L183-L190] , and Coinbase’s CDS with 100+ accessible components [22†L146-L154] [22†L157-

L160】 . Industry blogs on fraud and UI best practices also guided our approach 【36†L70-L74】 【20†L114-L123】 【59†L302-L310】 【44†L84-L91】 . All cited sources above are embedded for traceability.
